AI Engineering Fundamentals

Knowledge Base for RAG Assistant 0

1. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation is a pattern that combines information retrieval with a language model. Instead of relying only on the model's internal knowledge, the application first searches a knowledge base for relevant information and then provides that information to the model as context.

A basic RAG pipeline usually contains these steps:

• **Load:** Read source documents such as PDFs, text files, or web content.

• **Split:** Break documents into smaller chunks so retrieval can target specific passages.

• **Embed:** Convert chunks into numerical vectors called embeddings.

• **Store:** Save embeddings in a vector database such as Chroma.

• **Retrieve:** Search for chunks that are semantically similar to a user's question.

• **Generate:** Give the retrieved context to a language model and ask it to answer the question.

2. Embeddings and Vector Search

An embedding represents text as a vector of numbers. Texts with related meanings tend to have vectors that are close together in embedding space. A vector store can use these representations to retrieve relevant passages even when the user's wording is different from the wording in the source document.

For example, a question such as 'How does retrieval work?' may retrieve a passage containing 'The system searches the vector database for semantically similar document chunks.'

3. Tool Calling

Tool calling allows a language model to request an external function when it needs information or an action that should not be performed by the model alone. A tool can be a database search, calculator, API request, file lookup, or another application function.

In this project, the document retriever is exposed as a tool. The agent can decide that a user question requires searching the knowledge base, call the retriever, receive the results, and use those results to formulate an answer.

4. LangGraph

LangGraph is a framework for building stateful, multi-step agent workflows. A graph contains nodes that perform work and edges that determine what happens next.

A simple RAG agent can use a graph with a model node and a tool node. The model receives the conversation, decides whether to call the retrieval tool, and then receives the tool result before producing the final response.

5. MCP Fundamentals

The Model Context Protocol (MCP) is a standard for connecting AI applications to external tools and data sources. An MCP server can expose tools that an AI client can discover and call.

For RAG Assistant 0, the retrieval capability can be exposed through an MCP server. This demonstrates the fundamental MCP idea: a capability is made available as a tool with a defined name, description, and input schema.

6. How the Components Work Together

Component	Role
PDF knowledge base	Provides trusted source material.
Document loader	Reads the source document.
Text splitter	Creates manageable chunks.
Embedding model	Converts chunks into vectors.
Chroma	Stores and searches vectors.
Retriever	Finds relevant document chunks.
Tool calling	Lets the model request retrieval.
LangGraph	Controls the agent workflow.
MCP server	Exposes retrieval as a standardized tool.
Gemini	Uses retrieved context to generate the answer.

7. Example Questions

• What is RAG?

• What are the main steps in a RAG pipeline?

• Why are embeddings useful for document retrieval?

• What is tool calling?

• How does LangGraph control an agent workflow?

• What problem does MCP solve?

• How are MCP and tool calling related?

• What happens when a user asks a question that is not covered by the knowledge base?

8. Important Limitation

A RAG system is only as reliable as its retrieval and source material. If relevant information is not retrieved, the model may not have enough context to answer correctly. A good RAG assistant should therefore clearly state when the knowledge base does not contain enough information rather than

inventing unsupported facts.